Plan and Manage an Azure AI Solution (25-30%)

- **Foundry structure:** A Foundry resource contains isolated projects. Projects contain models, agents, tools, knowledge, deployments, and connections.
- **Portal and SDK:** Use `ai.azure.com` and `AIProjectClient` to manage Foundry project assets.
- **Specialized tools:** Use Language, Speech, Translator, Document Intelligence, and Content Understanding for focused tasks instead of an LLM when they provide a more reliable fixed result.
- **Model types:** A small language model (SLM) is cheaper and faster for simple, narrow tasks but weaker at deep reasoning than an LLM. An embedding model only turns text into vectors for vector search and does not generate answers.
- **Model selection:** The catalog holds over 10,000 models. Compare quality (quality index, higher is better), safety (attack success rate, lower is better), performance (latency, time to first token, generated and total tokens per second), cost, region support, modality, and fine-tuning support.
- **Grounding service choice:** Azure AI Search indexes private documents for keyword, vector, hybrid, and semantic retrieval. Web search or Grounding with Bing Search brings current public web information. File search grounds on uploaded files without a search service of your own.
- **Knowledge, memory, tools, guardrails:** Knowledge is the data an agent retrieves to ground answers. Memory is what the agent keeps about a user across conversations. Tools let the agent act or call other systems, and guardrails control safety.
- **Standard deployment:** Supports variable throughput without reserved capacity. Global Standard can route data outside a required region; Data Zone Standard keeps processing inside the US, EU, or APAC zone, also without reserved capacity. Global Provisioned requires reserved throughput.
- **Deployment data processing:** Global deployment types may process data in any Azure region. Data Zone types process only inside the zone (US, EU, or APAC). Standard and Regional Provisioned process inside the Azure geography of the resource.
- **Standard versus provisioned:** Standard types are pay-per-token and fit variable, bursty traffic without reserved capacity. Provisioned types reserve provisioned throughput units (PTUs) for predictable throughput and low latency variance. Batch types process large asynchronous jobs at lower cost with a 24-hour target.
- **Deployment type:** Serverless API deployments give real-time REST access and are billed by token usage or provisioned throughput units; they consume no virtual-machine vCPU quota. Batch is for offline work. Managed compute runs open-source and custom models on dedicated GPU capacity, billed per hour, with its own quota that is separate from VM quota.
- **Model version policy:** Choose **Once the current version expires** when stable model behavior matters more than immediate upgrades. **Upgrade once new default version becomes available** changes behavior soonest. **Opt out of automatic model version upgrades** needs manual upgrades and stops working when the version retires.
- **Model resources:** `OpenAI` creates Azure OpenAI resources; `CognitiveServices` creates a multi-service resource with one key and endpoint. `AIServices` creates a Foundry resource. An Azure Resource Manager `PUT` request creates or replaces a named resource.
- **Customer-managed keys:** Create an `OpenAI` resource and set `--encryption` JSON with `keySource: Microsoft.KeyVault` to protect data with a customer-managed key.
- **Project connections:** Centralize credentials and configuration so many apps or agents can reuse a managed connection. A connection stores the endpoint and credentials of a service once at project level, instead of per app.
- **Key Vault connection:** Use category `AzureKeyVault` with auth type `AccountManagedIdentity`. Without this connection, Foundry keeps connection secrets in a Microsoft-managed key vault; with it, the managed identity of the Foundry resource needs **Key Vault Secrets Officer** on your vault.
- **CI/CD evaluations:** GitHub Actions should use Azure Login with OIDC (permission `id-token: write`) to avoid long-lived credentials. The `microsoft/ai-agent-evals` action runs the evaluation; let the job fail below a pass-rate threshold so unsuitable pull requests cannot merge.
- **GitHub Actions project target:** The `microsoft/ai-agent-evals` action identifies the project with the input `azure-ai-project-endpoint`, set to the Foundry project endpoint; a tenant ID or model deployment name does not identify the project. Its other required inputs are `deployment-name`, `data-path`, and `agent-ids`.
- **Private networking:** Give the resource a private endpoint and disable public network access. Reach it from an on-premises network through a VPN or ExpressRoute, so confidential data never travels over the public internet.

© Rick Beerendonk

- **DefaultAzureCredential:** Uses Microsoft Entra managed identity in Azure and developer credentials locally for keyless authentication.
- **Least privilege:** Use **Foundry User** for inference on a Foundry resource and **Storage Blob Data Reader** to read blobs. Do not assign roles that start with "Cognitive Services" on a Foundry resource, except **Cognitive Services Usages Reader** to view quota; **Cognitive Services OpenAI User** is the inference role of a classic Azure OpenAI resource. A valid sign-in without such a data role gives HTTP 403.
- **Foundry roles: Foundry User** (earlier name: Azure AI User) is the least-privilege role to build in a project. **Foundry Agent Consumer** only allows calling agents.
- **Key Vault access:** A system-assigned managed identity needs the **Key Vault Secrets User** role to read secrets. This replaces a stored access key.
- **Publish an agent:** Every agent has a stable endpoint from the moment it is created. Publishing distributes the agent to Microsoft Teams and Microsoft 365 Copilot. The active version is "Always use latest" by default, so a new version is served at once; pin a version to protect production.
- **Agent identity:** A new agent gets its own Microsoft Entra agent identity when it is created, with no permissions. Assign the roles it needs to that identity, so its tool access is isolated and auditable. Only legacy agents share the identity of the project.
- **Keys:** Query keys are read-only; admin keys manage indexes and resources. Azure AI Search has two admin keys (primary and secondary) and up to 50 query keys.
- **Key rotation:** Switch the app to the secondary admin key, regenerate the compromised primary key, then switch the app back to the primary key. The two admin keys are regenerated one at a time; query keys are not regenerated but created and deleted on demand.
- **Simple service access:** An endpoint URI and subscription key need less administration than OAuth, SAS, or certificate-based identity. For production, use keyless access with Microsoft Entra ID.
- **Quotas:** Model quota is counted in tokens per minute (TPM) and requests per minute (RPM) per region, model, and deployment type. Going over the limit returns HTTP 429.
- **Rate limits:** Use exponential backoff with jitter for HTTP 429 responses; immediate retries usually fail again.
- **File limits:** An uploaded agent file can be at most 512 MB. Put large content in files and let file search retrieve the relevant parts, instead of sending it inside the request.
- **Tracing:** OpenTelemetry with Application Insights shows the ordered model calls, tool calls, tokens, and timing of one run. Nested parent-child spans show the hierarchy of a run, and span attributes hold tool call arguments and results.
- **Application Insights:** Required for Foundry Control Plane views of errors, runs, token usage, and traces; a Log Analytics workspace alone is not enough. Connecting it to the project also gives server-side traces without code changes.
- **Telemetry privacy:** Set `enable_content_recording=False` to exclude prompts, completions, and tool payloads from telemetry attributes. Content recording is on by default in `AzureAIOpenTelemetryTracer`.
- **Service name:** Give each app its own `OTEL_SERVICE_NAME` to tell its telemetry apart from other apps in the same Application Insights resource.
- **Diagnostics:** `RequestResponse` logs include request payloads, response information, status codes, and completions in Log Analytics. The other resource log categories are `Audit` and `Trace`.
- **Capacity signals:** Azure OpenAI AvailabilityRate shows model-side availability: the share of calls without a server error. Provisioned-managed Utilization V2 shows provisioned capacity pressure.
- **Cost signals:** Token usage metrics (Processed Prompt Tokens, Generated Completion Tokens) show whether input, output, or tool use drives cost.
- **Latency signals:** Time to Response measures the wait for the first output and Time to Last Byte the whole response. Do not use the old `Latency` metric for Azure OpenAI.
- **Quality monitoring:** The Agent Monitoring Dashboard shows token use, latency, success rate, and continuous evaluation results for production traffic. Groundedness scores show whether retrieved content still supports the answers; risk and safety metrics show policy violations. A quality score that falls over time on the same kind of traffic signals drift.
- **Search health:** Check indexer status and execution history for errors and warnings per run. Watch search latency, throttled query percentage, and storage use on the search service.

© Rick Beerendonk

- **Content Safety guardrails:** Configure checks for user input, output, tool responses, and tool calls; use **Block** when content must not continue. A guardrail is a set of controls, each with a risk, intervention points, and an action (annotate or block); tool call and tool response exist only for agents.
- **Blocking levels:** Harm categories are hate, sexual, violence, and self-harm, each with a severity. Blocking level **Low** blocks only high-severity content and is the least restrictive; **High** blocks low, medium, and high and is the most restrictive.
- **Prompt Shields:** Detect indirect prompt injection in retrieved content, documents, and user input. Image moderation does not detect hidden instructions or steganography. User prompt attack detection is on by default; document attack detection is off by default and must be enabled.
- **Spotlighting:** A preview option of the document attack control. It marks external documents as less trusted than system and user prompts, which reduces the effect of indirect injection. It works only through the Chat Completions API.
- **Fine-tuning safety checks:** Training data is checked for harmful content before training, and the tuned model is checked before deployment. The thresholds are fixed and not linked to your content filter settings, so a failed job is fixed by cleaning the data.
- **Safety evaluators:** Score hate, sexual, violence, and self-harm on a 0-7 severity scale against a threshold (default 3). The defect rate is the share of responses above the threshold, so a stricter threshold gives a higher rate.
- **Red teaming:** The AI Red Teaming Agent runs automated attack scans and scores the results with the safety evaluators.
- **Content Credentials:** Every image generated by Azure OpenAI image models carries a signed C2PA manifest that states its AI origin. Use it as provenance metadata.
- **Tool approval:** Set `require_approval` to `always` on an MCP tool to pause for an approval before each call; `never` skips it. `allowed_tools` limits which tools of the server the agent may use.
- **Workflows:** A sequential workflow gives deterministic order. An **Ask a question** node creates a human approval gate. Templates are Sequential, Group chat, and Human in the loop.
- **Session memory:** `scope` is the partition key of agent memory: a static value or `{{$userId}}`. How long memories are kept is set by a default time to live (TTL) on the memory store, not by a scope value.
- **User isolation:** `{{$userId}}` resolves to the authenticated user identity for separate per-user memory. It is read from the `x-memory-user-id` header or else from the caller's Microsoft Entra identity.
- **Responsible AI:** Fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability are the six principles.
- **Transparency:** Tell users how their data is processed and used.

© Rick Beerendonk

Implement Generative AI and Agentic Solutions (30-35%)

- **Direct model calls:** Use an Azure OpenAI endpoint for OpenAI-compatible Chat Completions and Responses APIs. Use a Foundry project endpoint for project agents, connections, and tools; `get_openai_client()` on the project client returns a client whose `responses.create()` calls a model or agent.
- **Chat state:** Chat Completions requires the app to resend the full message history. Responses API stores state server-side and uses `previous_response_id`.
- **Runtime components:** An agent holds the model, instructions, and tools. A conversation stores the history, and a response is one execution that produces output items.
- **Thread:** Threads and runs belong to classic agents. The current API replaces threads with conversations (the stored history) and runs with responses (one execution).
- **Conversation continuity:** Reusing a conversation ID restores messages, tool calls, and outputs across sessions. Storing only the last response or a summary does not.
- **Conversation:** The durable **Conversation** component preserves messages, tool calls, and tool outputs across turns and sessions.
- **Agent flow:** `agents.create_version()` with a `PromptAgentDefinition` -> `conversations.create()` -> `responses.create()` with the agent and conversation.
- **Get an agent:** Use `project_client.agents.get(agent_name=...)` to retrieve an existing agent.
- **System instructions:** Define the agent's role and response boundaries. Examples and sampling parameters do not set such boundaries.
- **LLMs:** Support detailed language generation and complex multi-step reasoning. SLMs can be less capable for deep reasoning.
- **Multimodal models:** Process text and images together.
- **Few-shot prompting:** Provide a small set of examples, usually 1-10, to guide output without retraining and without storing anything.
- **Zero-shot prompting:** Gives instructions without examples. Chain of thought asks for reasoning steps; it is not example-based learning.
- **Response completeness:** Increase `max_output_tokens` (Responses API) or `max_completion_tokens` (Chat Completions) when output is cut short; finish reason `length` shows that the limit was reached.
- **Adaptive reasoning:** A lower reasoning effort reduces latency and internal token use for simple tasks. OpenAI reasoning models such as the GPT-5 series use `reasoning_effort`; with reasoning switched on they reject `temperature` and `top_p`; Claude models use `thinking={"type": "adaptive"}` with `output_config={"effort": "low"}`.
- **Reflection and retry:** Let application code check a draft against the required content and regenerate when something is missing. Only scoring or blocking a weak answer does not improve it.
- **Optimization order:** Improve system prompts first, use RAG for missing current or private facts, and fine-tune for a consistent style or output format.
- **Fine-tuning:** Does not add current factual knowledge; use RAG for facts that change.
- **Model router:** A deployable model that sends each prompt to a suitable underlying model: small and cheap for simple requests, larger or reasoning models for complex ones. Its modes are Balanced (default), Cost, and Quality.

© Rick Beerendonk

- **Function tools:** The model requests a function call, but the host application executes the function and returns the result as `function_call_output`.
- **OpenAPI tools:** Define an API-key scheme under `securitySchemes` in the OpenAPI specification, with a matching `security` entry. The project connection supplies the key value: its key name must match the scheme, and the connection must be attached to the tool. Authentication is anonymous, API key, or project managed identity; per-operation headers are repetitive and a bearer scheme is the wrong type for an API key.
- **File search:** Grounds responses in uploaded documents and avoids sending oversized content directly in a request. It uploads files into a vector store and chunks and embeds them.
- **Code Interpreter:** Runs calculations and analysis in a sandbox.
- **Bing Grounding:** Retrieves current public web information. The web search tool does the same without a Bing connection.
- **Azure AI Search tool:** Takes a `project_connection_id` and an index. `query_type` defaults to `vector_semantic_hybrid` and `top_k` to 5.
- **`tool_choice="required"`:** Forces at least one tool call before the final answer. The values are `none` (no tools), `auto` (the model decides), and `required`.
- **MCP:** A standard protocol that connects agents to external services. The model can select discovered tools based on the task.
- **MCP tools:** Azure Language, Speech, and Document Intelligence can be exposed through MCP servers.
- **A2A:** The Agent2Agent tool lets a Foundry agent call a remote agent through the open A2A protocol.

© Rick Beerendonk

- **Persistent memory:** Storage-backed agent memory can retain user preferences across separate sessions. It is a memory store plus the memory search tool; pass tools as a list in the agent definition.
- **RAG:** Grounds model answers in retrieved authoritative content.
- **Agentic RAG:** Plans retrieval using conversation context, retrieves several chunks, and can run retrievals in parallel. Classic RAG is single-pass retrieval. In Azure AI Search this is agentic retrieval: a knowledge base lets an LLM plan subqueries and reranks the results.
- **Groundedness:** Measures whether the claims in a response are supported by the given context, without fabrication.
- **Relevance:** Measures whether the answer addresses the user query. Retrieval measures whether retrieved content matches the expected grounding context.
- **Retrieval evaluator:** Measures how relevant the retrieved chunks are to the query and needs no ground truth. Document Retrieval compares retrieved documents with ground-truth labels.
- **Response Completeness evaluator:** Measures whether the response covers all critical information of the ground truth (recall). Groundedness is the precision side.
- **Evaluators:** Risk and Safety metrics assess harmful content; AI Quality metrics assess groundedness and relevance. Risk and safety also covers protected material and indirect attacks, and NLP metrics such as similarity only compare text with a reference.
- **Protected material:** Is evaluated separately from harmful-content severity. It detects copyrighted text such as lyrics, recipes, and articles, as a yes-or-no check.
- **Agent evaluators:** Intent Resolution, Task Adherence, and Tool Call Accuracy judge whether an agent understood the request, followed its instructions, and called the right tools with the right parameters.
- **Workflow orchestration:** A workflow gives declarative, deterministic orchestration with branching and shared variables, without orchestration code. Group chat is for dynamic collaboration between agents. Foundry workflows retire on December 1, 2026 in favor of Microsoft Agent Framework.
- **Approval workflows:** `ask_question` pauses a YAML workflow for external approval; continue only when the approval value is `"approved"`.
- **Power Fx conditions:** `Not(IsBlank(Local.Var01))` continues only when a local answer exists. `{Upper(Local.Var01)}` interpolates that value in uppercase. Use the prefix `Local.` for local variables and `System.` for system variables.
- **LLM plus rules:** Keep fixed decisions such as validation, thresholds, and routing in deterministic code or workflow conditions, and use the model only for the language task. Rules give the same result every time; a prompt does not.
- **Orchestration:** Use concurrent for independent parallel work, sequential for fixed pipelines, group chat for collaboration, handoff for specialist ownership, and Magentic for manager-led adaptive planning.

© Rick Beerendonk

Implement Computer Vision Solutions (10-15%)

- **Multimodal messages:** Put text and images in the user message content array: `type: "input_text"` and `type: "input_image"` items in the Responses API, `type: "text"` and `type: "image_url"` items in Chat Completions. An image URL inside plain text is not seen as an image.
- **Image input:** Use a remote URL or a Base64 data URL such as `...` for a local image.
- **Captions:** A multimodal chat model writes captions; the prompt sets whether they are concise or detailed. Send several image items in one message to caption or compare several images.
- **Visual question answering:** Send the image with the question and instruct the model to answer only from what is visible.
- **Accessibility:** Generate short, accurate alt text that describes the visual evidence, and a longer description for complex images such as charts.
- **Image generation:** Image models create images from a text prompt. Controls include `size`, `quality`, and `output_format`.
- **Product fidelity:** Set `input_fidelity="high"` to preserve a reference product's recognizable visual characteristics. The default is `low`.
- **Inpainting:** Use a mask to edit only selected areas and preserve the rest of an image. Image variation changes the complete image. Send the image, a mask of the same size, and a prompt to the image edit endpoint.
- **Video generation:** `client.videos.create()` starts asynchronous generation; poll `client.videos.retrieve(video.id)` yourself until the status is `completed` or `failed`.
- **Video reference:** `input_reference` supplies one image as the first frame; it must match the output size exactly.
- **Video editing:** Remix makes targeted changes to an existing generated video and keeps its structure, motion, and framing. Pass the ID of the completed video instead of generating from scratch.
- **Content Understanding images:** An image analyzer (base `prebuilt-image`, or `prebuilt-imageSearch` for a description) extracts the fields you define, for example defects or product counts. It is not meant for images that are mainly text; use a document analyzer for those.
- **Video analysis:** A video analyzer (`prebuilt-video`, `prebuilt-videoSearch`) returns a transcript, key frames, and a description per segment. `enableSegment` with `contentCategories` splits the video into segments, and custom fields are generated per segment.
- **Content Understanding standard mode:** Processes single, independent files at lower cost.
- **Content Understanding pro mode:** Processes multiple files and supports cross-document reasoning, such as validating an invoice against a purchase order and contract. Standard and pro are modes of the `2025-05-01-preview` API; the `2026-06-01-preview` API adds an agentic mode, `config.workflow: "agentic"`, for multistep reasoning over one file.
- **Image moderation:** Classifies uploaded images for harmful-content severity; do not rely on OCR keyword scanning for this job. Send Base64 content or a blob URL to Content Safety. For a blob URL, the managed identity of the Content Safety resource needs **Storage Blob Data Contributor** or **Owner**.
- **Text in images:** Prompt Shields work on text. Extract the text with OCR and check it as a document with the document attack shield, optionally with Spotlighting. The user prompt shield, image moderation, and protected material detection do not detect instructions hidden in an image.
- **Visual policy rules:** Enforce brand, symbol, and watermark rules by combining content filters on prompts and generated images, Content Credentials for provenance, and a multimodal model or image analyzer that checks each image against your own rules, with human review for flagged images.

© Rick Beerendonk

Implement Text Analysis Solutions (10-15%)

- **Azure Language:** Provides language detection, sentiment, key phrases, summarization, named entity recognition, entity linking, and PII detection.
- **Sentiment:** Overall sentiment can be positive, negative, neutral, or mixed.
- **Tone and opinions:** Opinion mining (`show_opinion_mining=True`) adds sentiment per aspect, such as a positive room and a negative service in one review. For tone such as formal, angry, or urgent, prompt a language model and ask for a fixed label.
- **Summarization:** Extractive summarization returns the most important original sentences with a rank score. Abstractive summarization writes new sentences.
- **Structured outputs:** Give the model a JSON Schema so the output always follows it: `response_format` in Chat Completions, `text_format` with `responses.parse` in the Responses API. `strict: true` does the same for function arguments.
- **Domain tasks:** Shape LLM output for a domain with system instructions, a few examples, and a schema. Fine-tune only when a consistent style or format is still missing.
- **PII redaction:** The default policy `CharacterMask` replaces each character of a sensitive value with an asterisk. `EntityMask` replaces the value with its entity type, such as `[PERSON_1]` and `[EMAIL_1]`; there is no `Contact` super-category. The other policies are `SyntheticReplacement` and `NoMask`.
- **Content Safety:** `AnalyzeTextOptions(text=comment)` with `client.analyze_text()` classifies harmful-content severity, not PII.
- **Text limits:** A synchronous Azure Language request allows up to 5,120 characters per document. The number of documents per request depends on the feature, from 5 for NER and PII to 1,000 for language detection.
- **Real-time speech to text:** Processes streaming audio and returns intermediate results while the speaker talks, for live conversations and turn-taking.
- **Batch transcription:** Processes recordings after they finish; it is unsuitable for live turn-taking. Fast transcription returns the result for a file synchronously.
- **Speech SDK:** Use `SpeechConfig -> AudioConfig -> SpeechRecognizer -> recognize_once_async()` for speech to text.
- **Text to speech:** Use `SpeechConfig -> AudioOutputConfig -> SpeechSynthesizer -> speak_text_async()`.
- **Custom Speech:** REST calls require the custom speech **Project ID** in the `project` property. An expired custom model silently falls back to the latest base model for the same locale; batch transcription with an expired model fails with a 4xx error.
- **SSML:** Use `<phoneme>` to control the pronunciation of technical terms without training a custom voice. `<say-as>` sets how content such as dates is read, and `<prosody>` changes pitch and rate.
- **Realtime transports:** Use WebRTC for client-side web and mobile apps; it has the lowest latency for audio. Use WebSocket for server-to-server connections where low latency is less important, and SIP for telephony.
- **Voice Live:** Uses a `wss://` WebSocket for low-latency streaming. Stop playback when `input_audio_buffer.speech_started` signals barge-in. It is a managed speech-to-speech API with noise suppression, echo cancellation, and end-of-turn detection.
- **Audio input:** Audio-enabled chat models such as `gpt-audio` take audio directly in Chat Completions and reason over it, so no separate transcription step is needed.
- **Translator:** Supports text translation, document translation, transliteration, and more than 100 languages. The newest API version lets you choose between neural machine translation and an LLM.
- **Mixed-language input:** Language detection returns one language per text. Split text into single-language segments before translating for better accuracy.
- **Translation:** Converts meaning, for example Chinese text to English text. Transliteration converts a writing system into another script.
- **Speech translation:** Uses `SpeechTranslationConfig` to translate recognized speech into target languages.
- **Language resource networking:** A Language resource plus private endpoint plus VPN or ExpressRoute secures analysis of confidential on-premises documents.

glasspaper oblicum
© Rick Beerendonk

Implement Information Extraction Solutions (10-15%)

- **Azure AI Search pipeline:** Data source -> Indexer -> Skillset -> Index -> Search.
- **Indexer:** Connects a data source to a skillset for extraction and enrichment, then stores data in the search index.
- **Index-time enrichment:** Skills run when the indexer runs. Adding a skill has no effect until the indexer is run again.
- **Text Split:** Creates searchable chunks. The Azure OpenAI Embedding skill generates vectors for those chunks. Together they are called integrated vectorization.
- **Vector search:** Uses embeddings to match semantically similar wording, also when the query uses other words than the documents.
- **Hybrid search:** One request runs keyword search and vector search in parallel and merges the results with Reciprocal Rank Fusion. It serves exact names and codes as well as natural-language descriptions.
- **Semantic ranking:** Re-ranks an existing result set by meaning and can return captions and answers. It does not replace vector indexing.
- **Built-in skills:** OCR, NER, language detection, key phrases, sentiment, PII, translation, and image analysis enrich indexed content.
- **OCR skill:** Turns text in scanned documents and images into searchable text. Image Analysis describes images and does not read fields.
- **Custom skills:** The Custom Web API skill calls your own endpoint during indexing for enrichment that no built-in skill offers.
- **Content Understanding skill:** Extracts and chunks content in one skill, outputs tables and figures as Markdown, and treats a table that spans pages as one unit. `extractionOptions` adds images and location metadata for page-level citations.
- **Knowledge Store:** Saves enriched content in Azure Storage for downstream use.
- **Object projection:** Stores complete hierarchical JSON in Blob Storage. Table projection stores normalized rows in Azure Table Storage for analysis, and file projection stores images.
- **Document-level filtering:** Store allowed groups with documents, retrieve user group memberships, and filter queries with `groups/any(g: search.in(g, 'sales'))`. One index per group is not needed.
- **Search connections:** Configure Azure AI Search at Foundry project level to let several client apps use the same managed connection.
- **Embedded PDF images:** Use an indexer to extract images into `normalized_images` for OCR-ready data while retaining source-document citation links. Set `imageAction` to `generateNormalizedImages` on the indexer.
- **Structure-aware parsing:** Parsing that keeps headings, tables, and page numbers gives chunks that still carry structure and metadata. Plain-text parsing with fixed-size chunks loses them; Foundry Local calls the first option advanced data parsing.
- **Audio and video ingestion:** The Content Understanding RAG analyzers `prebuilt-audioSearch` and `prebuilt-videoSearch` turn media into transcripts, summaries, and descriptions that can be indexed.
- **Retrieval for agents:** Connect an index to an agent with the Azure AI Search tool, or build a Foundry IQ knowledge base that several agents share.
- **Content Understanding:** Combines OCR, layout analysis, and generative field extraction in one managed service, across document templates and without training a custom model.
- **Content extraction analyzers:** `prebuilt-read` gives text only. `prebuilt-layout` adds tables, sections, and structure. Neither needs a language model, and the `enableBarcode` setting reads barcodes and QR codes.
- **Document Search analyzer:** `prebuilt-documentSearch` creates RAG-optimized Markdown with layout, figure descriptions, and a summary; it uses a language model.
- **Custom analyzers:** Define field schemas with `extract`, `classify`, or `generate` methods. Route low-confidence fields, for example below 0.80, to human review. One schema extracts the same named fields from different layouts.
- **Generated fields:** Use field type `string` with method `generate` for AI-generated descriptions.
- **Field evidence:** `estimateFieldSourceAndConfidence` returns per-field confidence scores and source locations.
- **Document Intelligence:** Use `prebuilt-read` for text and `prebuilt-layout` for tables and document structure.
- **Markdown output:** Set `output_content_format=DocumentContentFormat.MARKDOWN` on the layout model to retain tables and sections for RAG (REST: `outputContentFormat=markdown`). Use `pages="1-3"` to analyze a PDF page range.

© Rick Beerendonk